

Kontraproduktívnosť princípov Clean Code

Marek Holik
Seminárna Práca | 2026

Univerzita Tomáša Baťu v Zlíne
FAI - Fakulta Aplikovanej Informatiky

Obsah

1	Úvod	5
2	Definícia Clean Code a súvisiacich pojmov.....	6
2.1	Kritika dogmatického uplatňovania <i>Clean Code</i>	6
2.2	”Clean Code”anti-patterny a bežné symptómy over-engineeringu	7
2.3	Meranie a pragmatizmus	7
3	Záver	8
3.1	Kedy je <i>Clean Code</i> kontraproduktívny	8
3.2	Praktické odporúčania.....	8
3.3	Príklady anti-patternov (rýchla kontrola).....	8
	Referencie.....	10

1 Úvod

Princípy čistého kódu, ako napríklad malé funkcie, zmysluplné pomenovania, minimalizácia komentárov a princíp DRY (Don't Repeat Yourself), sa stali súčasťou štandardnej výučby programovania a sú často považované za správny prístup k vývoju softvéru vo veľkom množstve spoločností a boli široko prijaté agilnými tímami po celom svete [3].

Pôvodným cieľom Clean Code bolo zvýšiť čitateľnosť, udržiavateľnosť a kvalitu softvéru. V priebehu rokov sa však objavili názory, že niektoré odporúčania a postupy boli prijaté príliš rýchlo a nie nutne overené komunitou programátorov. Existujú názory, že snaha o dokonalý kód môže viesť k predčasnej optimalizácii, zbytočnému pridávaniu funkcionalít a nadmernému navrhovaniu systémov (over-engineeringu).

Tento problém je obzvlášť aktuálny v prostredí agilného vývoja, kde je prioritou rýchle dodávanie hodnoty používateľovi. Silná orientácia na architektonickú čistotu môže spomaľovať vývoj, zvyšovať komplexnosť systému a odvádzať pozornosť od skutočných potrieb používateľov. Výskumná otázka tejto práce teda znie:

Vedie dogmatické uplatňovanie princípov Clean Code k predčasnej optimalizácii, preťažaniu funkcionalitami a over-engineeringu?

Na zodpovedanie tejto otázky bude analyzovaných zopár názorov odborníkov z praxe, akademických zdrojov a súčasných diskusií o softvérovom inžinierstve. Cieľom je identifikovať situácie, v ktorých Clean Code prináša hodnotu, a situácie, v ktorých sa môže stať kontraproduktívnym.

2 Definícia Clean Code a súvisiacich pojmov

Na začiatok je potrebné zdefinovať pojem "Clean Code". V literatúre sa tento pojem často spája s knihou: **Clean Code: A Handbook of Agile Software Craftsmanship** od Roberta C. Martina [3], kde autor prezentuje súbor princípov, pravidiel a odporúčaní, ktoré majú za cieľ zlepšiť čitateľnosť, udržiavateľnosť a kvalitu písaného kódu.

Medzi hlavné princípy Clean Code patrí:

- **Malé a zmysluplné funkcie:** Funkcie by mali byť krátke, robiť jednu vec a mať jasný názov, ktorý vystihuje ich účel.
- **Zmysluplné pomenovania:** Premenné, funkcie a triedy by mali mať názvy, ktoré jasne opisujú ich účel a zlepšujú čitateľnosť kódu.
- **Minimalizácia komentárov:** Kód by mal byť dostatočne čitateľný, aby nevyžadoval nadmerné komentovanie. Komentáre by mali byť použité len na vysvetlenie zložitých častí kódu alebo na poskytnutie kontextu.
- **Princíp DRY (Don't Repeat Yourself):** Opakujúci sa kód by mal byť refaktorovaný do samostatných funkcií alebo modulov, aby sa znížila duplicita a zvýšila udržiavateľnosť.

Téma Clean Code obsahuje aj súvisiace pojmy, ktoré sa často spomínajú v diskusiách o softvérovom inžinierstve a vývoji softvéru. Medzi tieto pojmy patrí:

- **Predčasná optimalizácia:** Situácie, keď sa vývojári snažia optimalizovať kód predtým, než je to potrebné, čo môže viesť k zložitejšiemu a menej čitateľnému kódu. Predtým, než začneme optimalizovať, je dôležité mať jasnú predstavu o tom, kde sú skutočné problémy s výkonom a či je optimalizácia naozaj potrebná. Mikro-optimalizácie malých funkcií môžu mať minimálny dopad na celkový výkon komplexného systému.
- **Preťažovanie funkcionalitami:** Pridávanie príliš veľa funkcií do systému, ktoré nie sú nevyhnutné pre jeho základnú funkcionalitu, čo môže znižovať jeho udržiavateľnosť. S týmto súvisí aj pojem "feature creep", ktorý označuje postupné pridávanie nových funkcií do softvéru, často bez ohľadu na pôvodný plán alebo potreby používateľov. Taktiež z hľadiska Clean Code je spájaný tento pojem s DRY princípom, pretože pridávanie nadbytočných funkcií môže viesť k zníženiu čitateľnosti kódu.
- **Over-engineering:** Nadmerné navrhovanie systémov s cieľom predvídať budúce potreby, čo môže viesť k zbytočne zložitým riešeniam. Aj keď Clean Code podporuje dobré návrhové princípy, nadmerné zameranie sa na architektonickú čistotu môže spôsobiť, že vývojári budú tráviť príliš veľa času navrhovaním riešení pre hypotetické scenáre.

2.1 Kritika dogmatického uplatňovania *Clean Code*

Niektorí autori a praktici upozorňujú, že princípy *Clean Code* by sa mali chápať ako usmernenie, nie ako absolútny zákon. Esej o "Clean" zdôrazňuje, že "čistota" kódu je kontextuálna

— čo je čitateľné a užitočné v jednom projekte, môže byť zbytočne komplikované v inom. Dôležité je hodnotiť prínos refaktoringu vzhľadom na dodávanú hodnotu a náklady na údržbu [1].

Konkrétne riziká dogmatického prístupu zahŕňajú zvýšenú kognitívnu záťaž, zbytočné vrstvy abstrakcie a skrytý technický dlh, ktorý vzniká pri kontinuálnom prepracovávaní riešení bez jasného merania benefitov [2].

2.2 "Clean Code" anti-patterny a bežné symptómy over-engineeringu

Nasledujúce scenáre často signalizujú, že snaha o "čistotu" prešla do prehnanej špecializácie a nadmerného navrhovania:

- **Vytvorenie microservisu pre workflow, ktorý patrí do jediného bounded contextu:** rozdeľovanie funkcionality bez dôkazov o samostatnej škálovateľnosti alebo nezávislom životnom cykle zvyšuje zložitosť infraštruktúry a komunikácie.
- **Pridanie plnohodnotnej state-management vrstvy pre stránku s niekoľkými komponentmi:** nadmerné nástroje pre malú DOM/UX logiku zvyšujú overhead a znižujú rýchlosť vývoja.
- **Budovanie plugin systému pre funkciu s jediným spotrebiteľom:** predikcia budúcich spotrebiteľov môže viesť k zbytočnej všeobecnosti a nákladom na testovanie a dokumentáciu.

Tieto anti-patterny majú spoločné príznaky: dlhšie plánovanie, viac infraštruktúrneho kódu, vyšší počet integračných bodov a znížená schopnosť rýchlo dodať hodnotu používateľovi. Autori, ktorí skúmajú skryté náklady "čistého" kódu, odporúčajú merať dopad takýchto rozhodnutí a preferovať pragmatické riešenia, kým sa nepreukáže potreba zložitosti [2].

2.3 Meranie a pragmatizmus

Praktický prístup odporúča: najprv merať (profilovanie, telemetria, používateľská spätná väzba), až potom optimalizovať alebo refaktoriť. Zameranie na rozhrania, kontrakty a testovateľné správanie často prináša výraznejší benefit než predčasné zdokonaľovanie implementácií [1], [2].

3 Záver

Dogmatické uplatňovanie princípov *Clean Code* môže viesť k predčasnej optimalizácii, preťaženiu funkcionalitami a over-engineeringu, ak sa tieto princípy používajú bez kontextu alebo bez merania prínosov. Keď sa kvalita kódu stáva vyššou prioritou než dodanie hodnoty používateľovi alebo než reálne potreby systému, vzniká riziko dlhšieho času vývoja a skrytého technického dlhu [2].

3.1 Kedy je *Clean Code* kontraproduktívny

- Ak sa princípy uplatňujú slepo bez ohľadu na kontext — výsledkom môže byť zbytočná komplexita a spomalenie vývoja.
- Ak sa prednostne rieši implementačná čistota namiesto návratu k používateľovi a merateľných metrik (výkon, stabilita, použiteľnosť).
- Ak architektúra obsahuje vrstvy a abstrakcie postavené na hypotézach o budúcich požiadavkách, ktoré sa nikdy nepreukážu.

3.2 Praktické odporúčania

1. **Najprv merajte:** Pred optimalizáciou alebo prepracovaním profilu a zistíte, kde je skutočné úzke miesto. Zamerajte úsilie na kritické 20
2. **Preferujte jednoduché rozhrania:** Chráňte kontrakty a rozhrania viac než implementácie — zmena implementácie je lacná, zmena rozhrania nie.
3. **Vyhňte sa predikciám:** Nevháňajte sa do generických riešení (plugin systémy, mikroservisy) bez dôkazov o potrebe.
4. **Automatizované kontroly:** Použite statickú analýzu, testy a architektonické kontroly na včasné odhalenie zápachov bez nadmerného manuálneho refaktorovania.
5. **Kultúra tímu:** Diskutujte trade-offs otvorene a rozhodujte podľa obchodných priorít, nie ideologických pravidiel [1].

3.3 Príklady anti-patternov (rýchla kontrola)

Ak pri revízii návrhu narazíte na nasledujúce návrhy, zvážte či nejde o over-engineering:

- Vytváranie microservisu pre workflow, ktorý patrí do jedného bounded contextu.
- Pridanie úplnej state-management vrstvy pre stránku s pár komponentmi.
- Budovanie plugin systému pre funkciu, ktorá bude mať pravdepodobne jediného spotrebiteľa.

Záver: *Clean Code* zostáva hodnotným súborom princípov — ak sa používa ako pragmatické usmernenie a dopĺňa ho meranie, automatizácia a dôraz na hodnotu používateľa. Ak sa však stane dogmou, môže brzdiť dodávky a vytvárať nečakané náklady. Preto odporúčame kombinovať zásady čistého kódu so zásadou "najprv merajte, potom refaktorujte" so zdravým skepticizmom voči predikciám prípadného rastu.

Referencie

- [1] S. **Hughes**, „Clean. “URL: <https://qntm.org/clean>
- [2] O. **Keswani**, „The Hidden Cost of “Clean Code”: When Over-Engineering Quietly Kills Shipping Velocity. “URL: https://dev.to/omieee_24/the-hidden-cost-of-clean-code-when-over-engineering-quietly-kills-shipping-velocity-1pg7
- [3] R. C. **Martin**, *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall, 2008, ISBN: 978-0-13-235088-4. URL: <https://www.lkhibra.ma/books/clean-code.pdf>